

[STANFORD NLP / DSPY FRAMEWORK](#)

DSPy: Programming - Not Prompting - Language Models

Stop hand-tuning prompt strings. Declare what your LM should do, compose calls into pipelines, and let a compiler optimize the prompts automatically.

April 2026 15 min read 3 interactive figures

The Alchemy Problem

Every LLM application starts the same way. You write a prompt. You test it. It doesn't work. You add "think step by step." You rearrange the instructions. You sprinkle in examples. Eventually it works - until you switch models and everything breaks.

This is prompt engineering: optimization by intuition. You are hand-tuning a string the same way early neural network practitioners hand-tuned features. It works for small problems, but it does not scale. Consider what happens when your pipeline has three LLM calls, each with its own prompt. Changing one prompt may break the others. Adding a new model requires re-tuning everything. Your "prompt library" becomes a collection of fragile, model-specific incantations.

DSPy shifts your focus from tinkering with prompt strings to programming with structured and declarative natural-language modules.

[DSPy](#), developed by Omar Khattab and the Stanford NLP group, proposes a different paradigm: stop writing prompts and start writing programs. Declare what each language model call should do, compose calls into pipelines, define a quality metric, and let a compiler figure out the prompts.

TRADITIONAL PROMPTING

- Hand-craft prompt strings
- Add examples manually
- Fragile to model changes
- Optimized by intuition
- Hard to maintain at scale

DSPY PROGRAMMING

- Declare typed signatures
- Compose reusable modules
- Portable across models
- Optimized by compiler
- Maintainable Python code

Three Pillars

DSPy is built on three core abstractions. Signatures declare input/output behavior. Modules implement strategies for calling LMs. Optimizers tune prompts and weights to maximize a metric. Together, they form a programming model for language models that mirrors how PyTorch works for neural networks.

SIGNATURE

`dspy.Signature`

Typed declaration of what your LM call should do. Inputs and outputs, not prompt strings.

MODULE

`dspy.Module`

Composable building blocks like Predict, ChainOfThought, and ReAct. Each carries learnable parameters.

OPTIMIZER

`dspy.Optimizer`

Algorithms that automatically tune prompts and weights. BootstrapFewShot, MIPROv2, and more.

Signatures: Declaring What, Not How

The fundamental unit in DSPy is the **signature** - a typed declaration of a language model's input/output behavior. Instead of writing a paragraph of instructions, you write a concise specification:

```
classify = dspy.Predict("sentence → sentiment: bool")
classify(sentence="it's a charming and often affecting journey.")
```

This single line declares: given a sentence, produce a boolean sentiment. No prompt template. No system message. No few-shot examples. Just the interface.

DSPy reads this signature and generates the actual prompt at runtime. It fills in field names, types, instructions, and formatting. When you later compile your program, the optimizer may add demonstrations, rewrite instructions, or even finetune weights - but your code stays exactly the same.

Inline signatures support type annotations for structured outputs:

```
# Basic question answering
qa = dspy.Predict("question → answer")

# RAG with context
rag = dspy.ChainOfThought("context, question → answer")

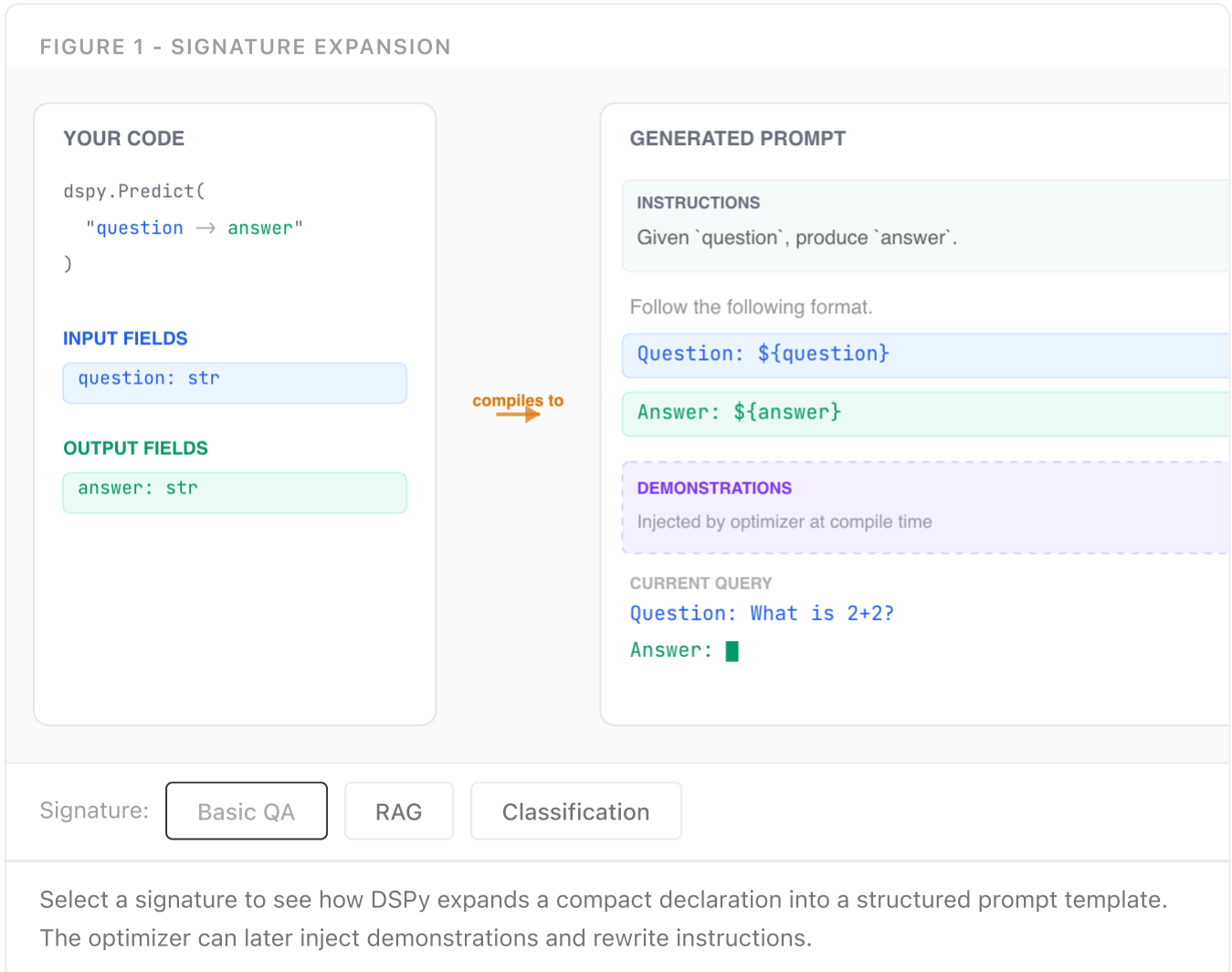
# Classification with constrained types
classify = dspy.Predict("sentence → sentiment: Literal['positive',
'negative', 'neutral']")
```

For complex tasks, class-based signatures add descriptions and field-level constraints:

```
class CheckFaithfulness(dspy.Signature):
    """Verify that the text is grounded in the provided context."""
    context: str = dspy.InputField(desc="facts assumed to be true")
    text: str = dspy.InputField()
    faithfulness: bool = dspy.OutputField()
```

Key insight: A signature is a contract, not a prompt. It says what the LM should accomplish. How it accomplishes it - the actual prompt text, demonstrations, and formatting - is determined at compile time by the optimizer.

FIGURE 1 - SIGNATURE EXPANSION



Modules: Building Blocks for LM Programs

Signatures declare what to do. Modules decide *how* to do it.

`dspy.Predict` is the simplest module - it wraps a signature and calls the LM directly.

`dspy.ChainOfThought` does the same thing but automatically injects a `reasoning` step before the output. The LM must show its work before answering. `dspy.ReAct` goes further, enabling the LM to use external tools.

Not every module calls an LM. `dspy.Retrieve` connects to a retrieval index - a vector database, a ColBERT index, or any search backend you configure. You set it up once via `dspy.configure(rm=your_retriever)`, and then `dspy.Retrieve(k=3)` fetches the top-`k` relevant passages for a given query. It returns a `Prediction` object with a `.passages` field - a list of strings you can pass to downstream modules as context.

The real power comes from composition. Inherit from `dspy.Module`, define sub-modules in `__init__`, and wire them together in `forward`. Here is a retrieval-augmented generation pipeline that combines `Retrieve` with `ChainOfThought`:

```
# Configure a retrieval model (once, at startup)
# dspy.configure(lm=lm, rm=ColBERTv2(url="..."))

class RAG(dspy.Module):
    def __init__(self, num_docs=3):
        self.retrieve = dspy.Retrieve(k=num_docs) # fetches top-k passages
        self.generate = dspy.ChainOfThought("context, question → answer")

    def forward(self, question):
        context = self.retrieve(question).passages # list[str]
        return self.generate(context=context, question=question)
```

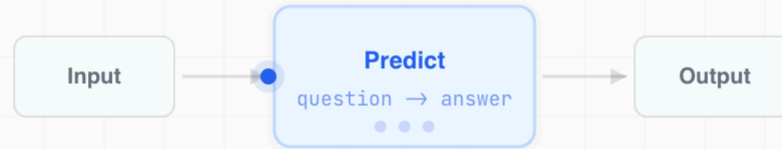
The data flow here is: `question` goes into `Retrieve`, which queries the index and returns relevant passages. Those passages become the `context` input for `ChainOfThought`, which reasons over them and produces an `answer`. `Retrieve` handles search; the LM never sees the full corpus, only the passages the retriever selects.

This pattern is deliberately reminiscent of PyTorch. `nn.Module` becomes `dspy.Module`. Layers become LM calls (and retrieval calls). The `forward` method defines data flow. Each module carries learnable parameters - not neural weights, but prompt instructions and demonstrations that the optimizer can tune.

You can use standard Python control flow - loops, conditionals, recursion - inside `forward`. DSPy traces the LM calls at compile time, so the optimizer can reach into any module in the pipeline.

FIGURE 2 - MODULE PIPELINE

dspy.Module



Each module carries learnable parameters (instructions + demonstrations)

Pipeline:

Predict

Chain of Thought

RAG Pipeline

Multi-Hop

Explore different module compositions. Data flows left to right through each module. More complex pipelines chain multiple LM calls, each with its own learnable parameters.

The Optimizer: Compiling Your Program

Here is where DSPy diverges from everything else in the LLM ecosystem. You do not manually tune your prompts. You define a **metric** and let an **optimizer** do it for you.

The simplest optimizer, `BootstrapFewShot`, works in three steps:

1. Run your unoptimized pipeline on each training example
2. Check if the output passes your metric function
3. Collect passing input/output traces as few-shot demonstrations

The optimized program now includes these demonstrations in its prompts. The LM sees concrete examples of successful behavior, not just abstract instructions.

```
def metric(example, prediction, trace=None):
    return example.answer.lower() == prediction.answer.lower()
```

```
optimizer = dspy.BootstrapFewShot(metric=metric, max_bootstrapped_demos=4)
optimized_rag = optimizer.compile(RAG(), trainset=trainset)
```

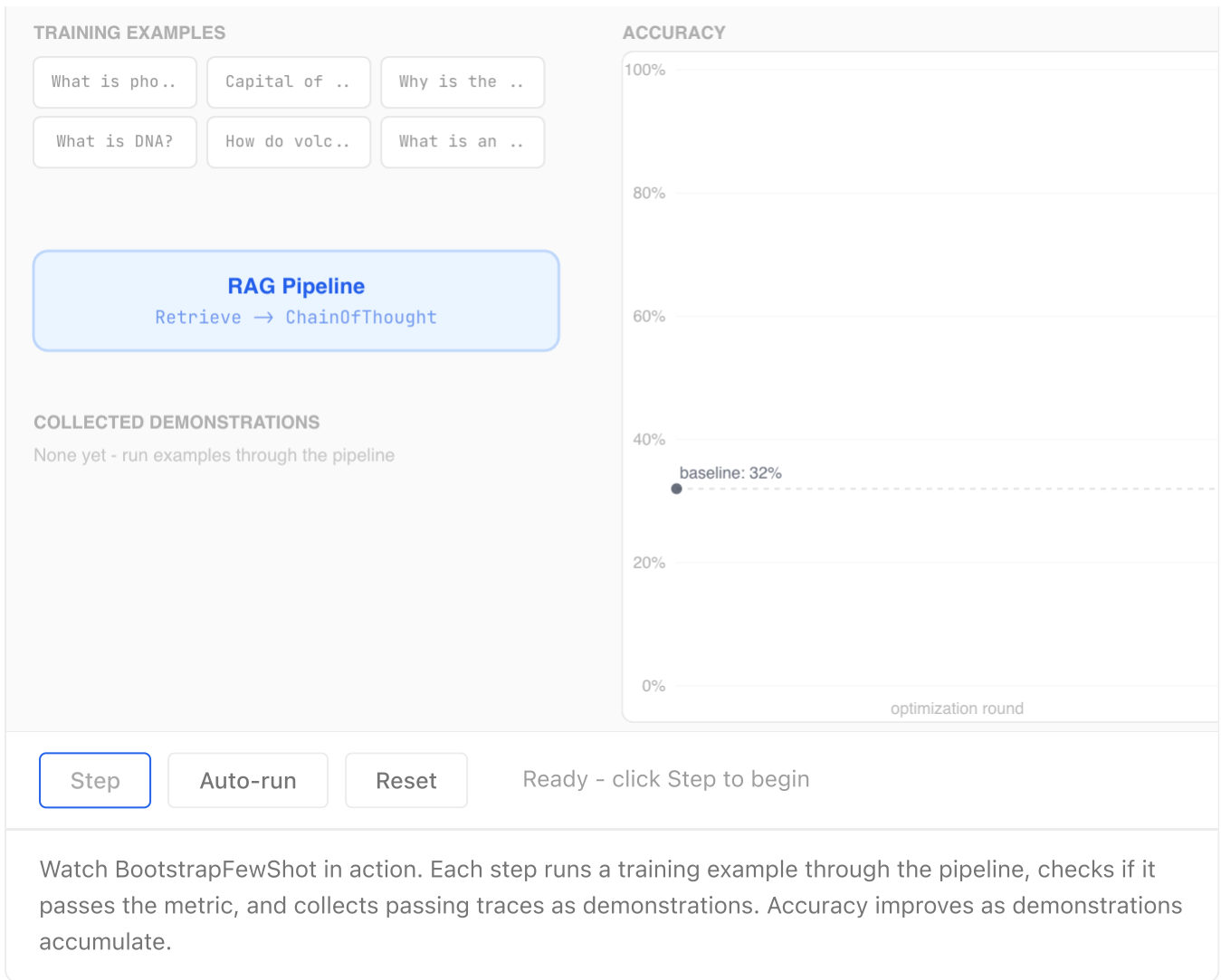
MIPROv2 goes further. It generates candidate instructions for each module, synthesizes demonstrations, and uses Bayesian optimization to search the combined space of instructions and examples. A typical run costs about \$2 and takes 10 minutes - but can improve accuracy by 10-40%.

The compilation analogy: Calling `optimizer.compile()` is like calling `model.fit()` in scikit-learn. You hand it your program and training data. It returns an optimized version with tuned prompts. Your code does not change - only the generated prompts improve.

Choosing an Optimizer

~10 examples	BootstrapFewShot - fast, simple, good baseline
50+ examples	BootstrapFewShotWithRandomSearch - tries multiple seeds
200+ examples	MIPROv2 - instruction + demo optimization via Bayesian search
Cost-sensitive	BootstrapFinetune - distill into a smaller finetuned model

FIGURE 3 - BOOTSTRAPFEWSHOT OPTIMIZER



What the compiled prompt looks like: After optimization, the collected demonstrations are injected directly into the prompt. A compiled RAG module's prompt might look like this:

```
# Compiled prompt for: context, question → answer
```

```
---
```

```
Follow the following format.
```

```
Context: ${context}
```

```
Question: ${question}
```

```
Reasoning: Let's think step by step.
```

```
Answer: ${answer}
```

```
---
```



```
# Demo 1 (bootstrapped from training data)
```

```
Context: "France is a country in Western Europe. Its capital is Paris..."
```

```
Question: "What is the capital of France?"
```

```
Reasoning: "The context states that France's capital is Paris."
```

```
Answer: "Paris"
```

```
# Demo 2 (bootstrapped from training data)
```

```
Context: "DNA, or deoxyribonucleic acid, carries genetic instructions..."
```

```
Question: "What is DNA?"
```

```
Reasoning: "The context defines DNA as a molecule carrying genetic info."
```

```
Answer: "A molecule that carries genetic instructions for development"
```

```
---
```

```
# Current query
```

```
Context: "The aurora borealis occurs when charged particles from the sun..."
```

```
Question: "What causes the Northern Lights?"
```

```
Reasoning: _
```

The demonstrations teach the LM by example. Instead of hand-writing "answer concisely based on the context," the optimizer found real input/output pairs that passed the metric and injected them as few-shot examples. The LM sees what good behavior looks like, not just a description of it.

Putting It Together

Here is a complete DSPy program: a retrieval-augmented generation pipeline that answers questions using a corpus of documents. In traditional prompt engineering, this would require crafting separate prompts for retrieval re-ranking, reasoning, and answer generation. In DSPy, it is a few lines of Python:

```
import dspy
```

```
# 1. Configure the LM and retrieval backend
```

```
lm = dspy.LM("openai/gpt-4o-mini")
```

```
rm = dspy.ColBERTv2(url="http://localhost:8893/search")
```

```
dspy.configure(lm=lm, rm=rm)
```

```
# 2. Define the program
```

```

class RAG(dspy.Module):
    def __init__(self):
        self.retrieve = dspy.Retrieve(k=3)
        self.generate = dspy.ChainOfThought("context, question → answer")

    def forward(self, question):
        context = self.retrieve(question).passages
        return self.generate(context=context, question=question)

# 3. Define the metric
def answer_match(example, pred, trace=None):
    return example.answer.lower() in pred.answer.lower()

# 4. Compile
optimizer = dspy.BootstrapFewShot(metric=answer_match,
max_bootstrapped_demos=4)
optimized_rag = optimizer.compile(RAG(), trainset=trainset)

# 5. Use it
result = optimized_rag(question="What causes the Northern Lights?")
print(result.answer)

```

Five steps. No prompt strings anywhere. The optimizer saw your training examples, ran them through the pipeline, collected the ones that worked, and injected them as demonstrations. If you later switch from GPT-4o-mini to Claude or Llama, you re-run `compile()` and the optimizer adapts the prompts to the new model's strengths.

You can save and load optimized programs:

```

# Save the optimized program
optimized_rag.save("optimized_rag.json")

# Load it later
loaded_rag = RAG()
loaded_rag.load("optimized_rag.json")

```

The saved file contains the optimized instructions and demonstrations in JSON format - no model weights, just the learned prompting strategy.

Why This Matters

The shift from prompting to programming mirrors what happened in deep learning. Early practitioners hand-designed features - edge detectors, color histograms, texture descriptors. Backpropagation made features learnable. The field exploded.

DSPy is attempting the same transition for LLM applications. When prompts are generated by a compiler rather than a human, several things change:

Programs become portable. The same DSPy code can target GPT-4, Claude, or a local Llama model. The optimizer adapts to each model's strengths - more demonstrations for smaller models, more concise instructions for larger ones.

Optimization becomes systematic. Instead of "try adding 'think step by step' and see what happens," you define a metric, provide examples, and let an algorithm search the space. This is repeatable, measurable, and debuggable.

Complexity becomes manageable. A pipeline with five LM calls is not five prompt engineering problems. It is one program with five modules. The optimizer tunes them jointly, accounting for how changes in one module affect downstream modules.

DSPy does not eliminate the need to think carefully about your task. You still need good signatures, a meaningful metric, and representative training data. But it moves the work from "guess the right prompt" to "specify the right behavior" - from craft to engineering.

The key question is not "what prompt should I write?" but "what should this module do, and how will I know if it's doing it well?"

Built by analyticalmonk. Inspired by distill.pub. DSPy by Omar Khattab and Stanford NLP.